Hes·so VALAIS WALLIS
Haute Ecole d'Ingénierie
Hochschule für Ingenieurwissenschaften

Nom : _____

Prénom : _____

Classe : _____

# Examen intermédiaire 2 – SOLUCE -

# Embedded programming – 17/12/2025

| Question / points | 1 / 3 | 2 / 3 | 3 / 8 | 4 / 8 | 5 / 14 | Total / 36 |
|---|---|---|---|---|---|---|
| Résultat | | | | | | |

***Consigne :***

Lisez attentivement la donnée et répondez de manière **lisible** aux questions. Le cours et la documentation de laboratoire sont autorisés.

### Question 1 (le STM32): (3 points)

Répondez aux questions :

Le watchdog sert à générer des impulsions de temps au système (HAL_Tick) ? FAUX

L'entrée GPIO_EXTI_2 est une interruption plus prioritaire que GPIO_EXTI_5 ? FAUX

Dans quel cas, par exemple, une interruption « Hard Fault » peut survenir ? Bad instruction, memory access

Notre STM32U5G9 possède un convertisseur A/D de 14 bits.

Le bit de parité de l'UART sert à contrôler si une erreur de transmission est apparue ? VRAI

Le STM32 possède 14 timers différents.

### Question 2 (le C): (3 points)

Répondez aux questions :

Le code `uint32_t * myPtr = count;` crée un pointeur qui pointe sur count ? FAUX

La taille d'un pointeur dépend du type pointé ? FAUX

Pour accéder à la valeur pointée par un pointeur, le caractère **&** doit être utilisé? FAUX

La « fonction » `sizeof()` retourne la taille d'une variable en nombre de bytes ? VRAI

Le code suivant `const uint8_t msg[] = "Yopla";` occupe 5 bytes en mémoire Flash ? FAUX, 6

Le printf suivant `printf("Res :%d / %d = %d",9,2,4.5);` va afficher : Res :9 / 2 = 4

**Question 3: (8 points)**

Ecrivez les functions LifoCreate, LifoPut et LifoGet en utilisant uniquement les variables données ci-dessous.
La fonction LifoCreate va créer l'espace mémoire à l'aide d'une allocation mémoire dynamique de la taille demandée. Elle retournera la taille demandée si tout s'est bien passé, sinon elle retournera -1.
La fonction LifoPut va placer dans le buffer la valeur passée en paramètre. Elle retournera -1 si le buffer est plein et que la valeur n'a pas pu être placée. Sinon, elle retournera le nombre d'éléments dans le buffer.
La fonction LifoGet placera dans la variable passée par référence la **dernière** valeur placée dans le buffer**. LiFo signifie Last-In First-Out**. Si le buffer était vide, cette fonction retournera -1, sinon, elle retournera le nombre d'éléments restant dans le buffer (après l'appel à LifoGet()).

```
uint32_t * bufferPtr;
uint8_t bufferOffset=0;
uint8_t bufferSize;

int8_t LifoCreate(uint8_t size)
{
  bufferSize = size;
  bufferPtr = malloc(size * 4)
  if(bufferPtr != null)
  {
    return size;
  }
  return -1;
}

int8_t LifoPut(uin32_t value)
{
  if((bufferOffset) == bufferSize)
  {
    return -1;
  }
  buffePtr[bufferOffset] = value;
  bufferOffset++;
  return bufferOffset;



}

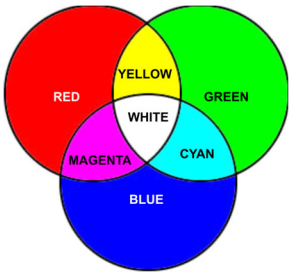
int8_t LifoGet(uin32_t * value)
{
  if((bufferOffset) == 0)
  {
    return -1;
  }
  *value = buffePtr[bufferOffset-1];
  bufferOffset--;
  return bufferOffset;
}  // utilisez une feuille supplémentaire si nécessaire
```

## Question 4: (8 points)

Soit une led RGB (RED, GREEN, BLUE) sur laquelle nous désirons afficher la séquence visible ci-dessous. Cette séquence s'incrémentera d'une position à **chaque pression** sur le bouton. Arrivé à la fin, la séquence recommencera.

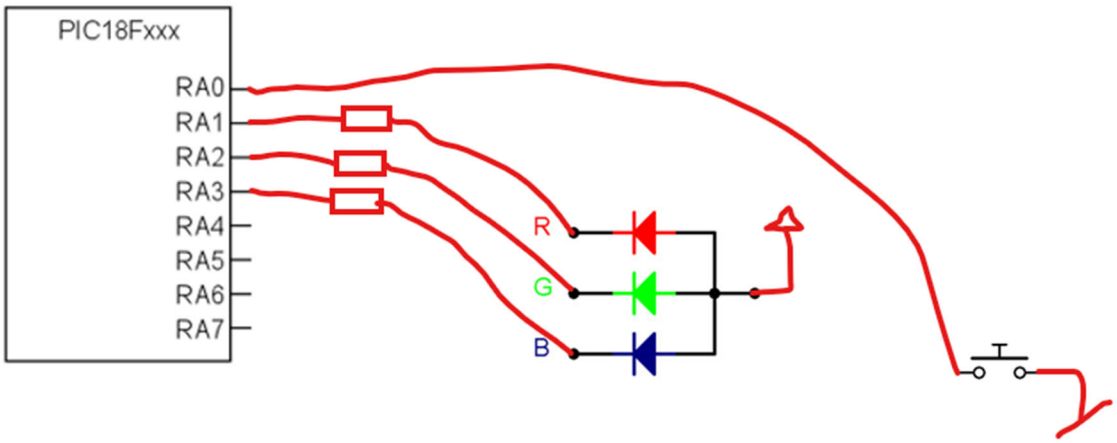
| YELLOW | >> | CYAN | >> | MAGENTA |
|---|---|---|---|---|

Pour information, voici le système d'addition des couleurs permettant d'obtenir les couleurs secondaires à partir des primaires.



Additive colors

Complétez, le schéma ci-dessous, les connections du bouton et de la led RGB. Ajoutez des résistances (sans valeurs à calculer) et des VCC et GND si nécessaire.




Ecrivez le code réalisant ce programme. Vous utiliserez CubeMX pour l'initialisation des entrées (le bouton **sans** interruption) et des sorties (les 3 leds). Cela implique donc que la première partie du main vous est fournie ci-dessous. On suppose que le bouton n'a pas de rebonds.
Les noms des GPIO seront : LED_R, LED_G, LED_B et BTN, toutes connectées sur le port A.

```
uint32_t main(void)

{
  HAL_Init();
  SystemClock_Config();
  SystemPower_Config();
  MX_GPIO_Init();
  uint8_t count = 0;
  HAL_GPIO_Write(PORTA, LED_R, GPIO_RESET);
  HAL_GPIO_Write(PORTA, LED_G, GPIO_RESET);
  HAL_GPIO_Write(PORTA, LED_B, GPIO_SET);
```

```
  while (1)
  {
    while(HAL_GPIO_Read(PORTA, BTN) == GPIO_SET){}; // wait press
    count=(count+1) % 3;
    switch(count)
    {
      case 0:
        HAL_GPIO_Write(PORTA, LED_R, GPIO_RESET);
        HAL_GPIO_Write(PORTA, LED_G, GPIO_RESET);
        HAL_GPIO_Write(PORTA, LED_B, GPIO_SET);
      break;
      case 1:
        HAL_GPIO_Write(PORTA, LED_R, GPIO_SET);
        HAL_GPIO_Write(PORTA, LED_G, GPIO_RESET);
        HAL_GPIO_Write(PORTA, LED_B, GPIO_RESET);
      break;
      case 2:
        HAL_GPIO_Write(PORTA, LED_R, GPIO_RESET);
        HAL_GPIO_Write(PORTA, LED_G, GPIO_SET);
        HAL_GPIO_Write(PORTA, LED_B, GPIO_RESET);
      break;
    }
    while(HAL_GPIO_Read(PORTA, BTN) == GPIO_RESET){}; // wait released









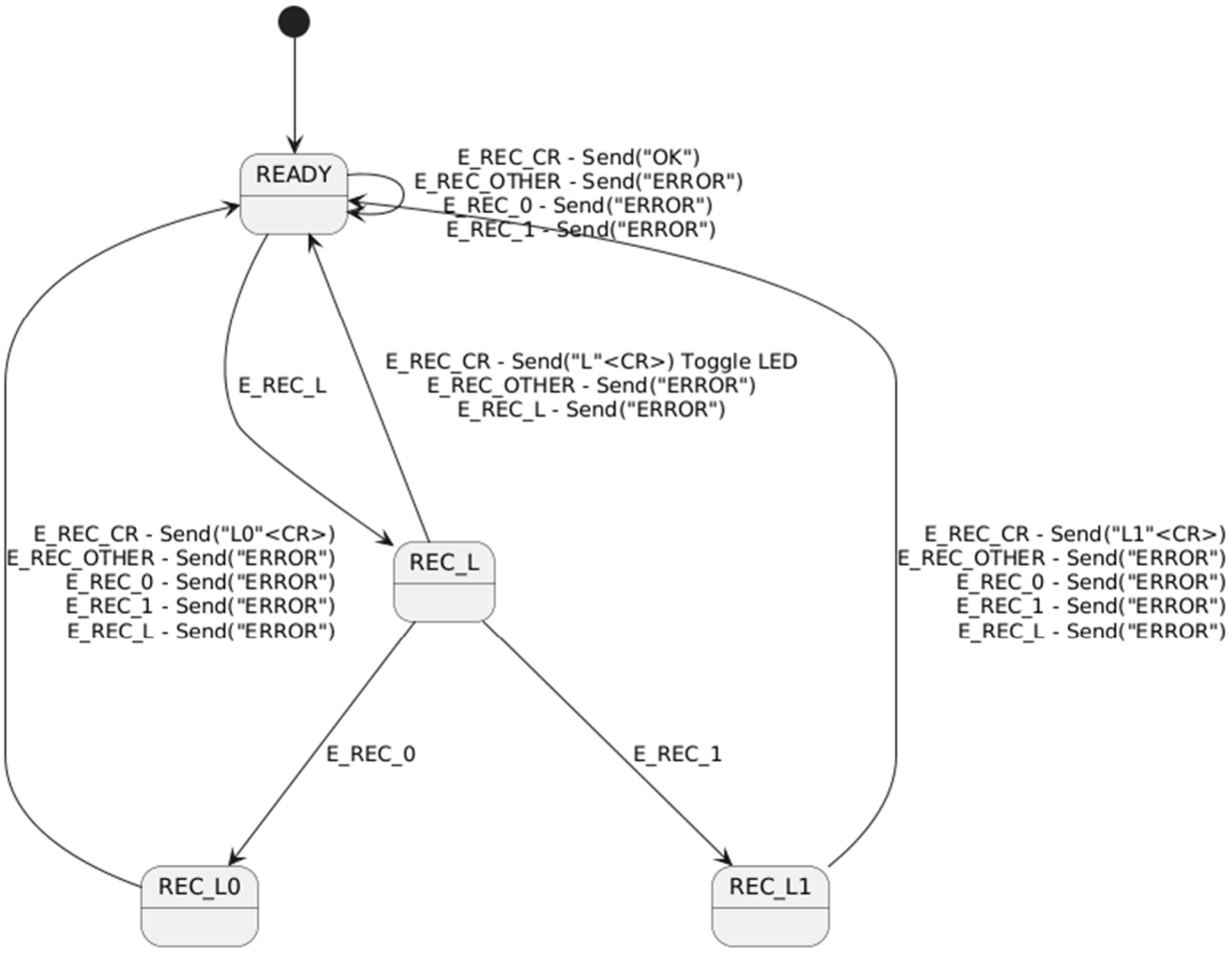
  }
}
```

**Question 5: (14 points)**

Nous désirons développer un système qui reçoit des commandes depuis un port série UART d'un PC et qui exécute certaines actions en fonction des commandes reçues. Pour cela, un certain nombre de commandes seront implémentées.

| Command name | ASCII string | Description | Actions |
|---|---|---|---|
| STATUS | <CR> | Is system ok | Answer "OK<CR>" |
| LED_TOGGLE | L<CR> | Toggle LED | Toggle LED,<br>Answer "L<CR>" |
| LED_SET | L0<CR> (x=0 for off)<br>L1<CR> (x=1 for on) | Set LED or clear LED | Set LED as expected<br>Answer "L0<CR>" or<br>Answer "L1<CR>" |
| ERROR | Any other than above | Unknow command | Answer "ERROR<CR>" |

Pour ce programme, nous utiliserons naturellement le XF. Pour vous aider, voici la liste des évènements possibles (E_REC_CR, E_REC_L, E_REC_0, E_REC_1, E_REC_OTHER). Ces évènements seront postés par l'interruption de l'UART en fonction des caractères reçus. Veuillez compléter la machine d'état ci-dessous avec toutes les transitions et actions nécessaires. Attention, pensez à tous les cas d'erreurs.



A l'aide de la machine d'état dessinée précédemment, complétez le code de l'interruption de l'UART permettant d'envoyer des évènements à notre processus de la machine d'état.

```
uint8_t recByte;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  uint8_t tmid;                    // not used but mandatory
  Event evt;                       // the event to push
  switch(recByte)
  {
    case 0x0D:          // is the CR

      evt.delay = 0;
      evt.id = E_REC_CR;              // id example
      evt.processEvent = processAnalyse;  // function to call example
      XF_pushEvent(evt,true,&tmid);   // true because in ISR
    break;
    case 'L':
      evt.delay = 0;
      evt.id = E_REC_L;              // id example
      evt.processEvent = processAnalyse;  // function to call example
      XF_pushEvent(evt,true,&tmid);   // true because in ISR
    break;
    case '0':
      evt.delay = 0;
      evt.id = E_REC_0;              // id example
      evt.processEvent = processAnalyse;  // function to call example
      XF_pushEvent(evt,true,&tmid);   // true because in ISR
    break;
    case '1':
      evt.delay = 0;
      evt.id = E_REC_1;              // id example
      evt.processEvent = processAnalyse;  // function to call example
      XF_pushEvent(evt,true,&tmid);   // true because in ISR
    break;
    default:
      evt.delay = 0;
      evt.id = E_REC_OTHER;              // id example
      evt.processEvent = processAnalyse;  // function to call example
      XF_pushEvent(evt,true,&tmid);   // true because in ISR
    break;









  }
        HAL_UART_Receive_IT(&huart2, &recByte, 1);
}
```

Pour finir, compléter le code du processus de la machine d'état.

```
bool processAnalyse(Event *ev)

typedef enum
{
  INIT,
  READY,
  REC_L,
  REC_L0,
  REC_L1


}State;
static State state=INIT;
static State oldState=INIT;

switch(state)                        // transition states and on exit actions
{
  case INIT:
    State = READY;
  break;
  case READY:
    if(ev->id == REC_CR)
    {
      HAL_UART_Transmit(&huart2,"OK\r",3,10);
    }
    else if(ev->id == REC_L)
    {
      state = REC_L;
    }
    else
    {
      HAL_UART_Transmit(&huart2,"ERROR\r",6,10);
    }
  break;
  case REC_L:
    if(ev->id == REC_0)
    {
      state = REC_L0;
    }
    else if(ev->id == REC_1)
    {
      state = REC_L1;
    }
    else if(ev->id == REC_CR)
    {
      state = READY;
      HAL_GPIO_Toggle(LED_PORT, LED_PIN);
      HAL_UART_Transmit(&huart2,"L\r",2,10);
    }
    else
    {
      HAL_UART_Transmit(&huart2,"ERROR\r",6,10);
      state = READY;
    }

  break;
```

```
  case REC_L0:
    if(ev->id == REC_CR)
    {
      HAL_GPIO_Write(LED_PORT, LED_PIN, GPIO_RESET);
      HAL_UART_Transmit(&huart2,"L0\r",3,10);
      state = READY;
    }
    else
    {
      HAL_UART_Transmit(&huart2,"ERROR\r",6,10);
      state = READY;
    }

  break;


  case REC_L1:
    if(ev->id == REC_CR)
    {
      HAL_GPIO_Write(LED_PORT, LED_PIN, GPIO_SET);
      HAL_UART_Transmit(&huart2,"L1\r",3,10);
      state = READY;
    }
    else
    {
      HAL_UART_Transmit(&huart2,"ERROR\r",6,10);
      state = READY;
    }

  break;
```

```
}
if (state == oldState)              // on loop action (nothing to do)
{
  return 0;
}
oldState = state;
//                                  // on entry action (nothing to do)
```